

TABLE OF CONTENTS

- 1. Abstract**
- 2. Acknowledgement**
- 3. Introduction**
- 4. Company Profile**
- 5. Overview of the Organization**
- 6. Objectives of the Internship**
- 7. Role and Responsibilities**
- 8. Tools and Technologies Used**
- 9. Project Work / Tasks Performed**
- 10. Methodology / Working Process**
- 11. Implementation Details**
- 12. Source Code**
- 13. Results**
- 14. Output**
- 15. Learning Outcomes**
- 16. Skills Gained**
- 17. Challenges Faced**
- 18. Conclusion**
- 19. Future Scope**
- 20. References**

1. ABSTRACT

This report presents the details of my internship completed at TVK Technologies from 10 June 2026 to 10 July 2026, in the field of Data Analytics and Artificial Intelligence. The main objective of this internship was to gain practical knowledge of data science techniques and understand how predictive models support real-world business decision-making, particularly in the financial services sector.

During the internship, I was trained in the fundamentals of Data Science, including data cleaning, exploratory data analysis (EDA), feature engineering, and machine learning based classification. As a hands-on project, I built and evaluated a complete Loan Approval Prediction System using a real-world bank loan applicant dataset, using Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.

The project involved cleaning and preparing 4,269 loan applications, performing exploratory analysis to understand what drives loan approval, engineering financially meaningful ratio-based features, and training and comparing three classification models (Logistic Regression, Decision Tree, and Random Forest) to identify the best performer for predicting loan approval outcomes.

Beyond the technical project, the internship strengthened my understanding of the complete data science workflow, from raw data to a deployable predictive model, and improved my skills in problem solving, model evaluation, and communicating analytical results clearly. Overall, this internship provided valuable industry exposure and has strengthened my technical foundation for a future career in Data Science and Artificial Intelligence.

2. ACKNOWLEDGEMENT

I express my sincere gratitude to TVK Technologies for providing me with the opportunity to undergo internship training in AI with Data Science.

I thank my internship mentor, Program Manager Mr. Priyatharshan A, and team members for their valuable guidance, continuous support, and encouragement throughout the internship period.

I also express my heartfelt gratitude to the Principal, Head of the Department, faculty members, and my project guide of University College of Engineering, Tindivanam, for their motivation and encouragement.

Finally, I thank my parents and friends for their constant support and encouragement throughout my internship.

3. INTRODUCTION

Data Science combines statistics, programming, and machine learning to extract meaningful insights from data and solve real-world problems.

Understanding customer purchase behavior is one of the major challenges for e-commerce and retail businesses. Without knowing which customers are likely to buy, marketing budgets are spent inefficiently, discounts are offered to customers who would have purchased anyway, and genuinely interested customers are left without the right nudge at the right time.

The primary objective of this internship project was to build a Customer Purchase Prediction System capable of analyzing customer demographic and behavioral data, predicting the likelihood of purchase using machine learning, and automatically recommending a personalized marketing strategy using an AI-based decision layer.

The project involved collecting and exploring a customer purchase dataset, building a preprocessing and modelling pipeline, training and evaluating a classification model, and converting model predictions into actionable, prioritized marketing recommendations.

4. COMPANY PROFILE

TVK Technologies is an Information Technology company specializing in software development, Data Science, Machine Learning, Artificial Intelligence, Python development, and web application development. The company focuses on delivering innovative and reliable software solutions that address real-world business challenges using modern technologies.

TVK Technologies provides end-to-end software development services, including application design, development, testing, deployment, and maintenance. It also offers internship and training programs that help students gain practical knowledge through hands-on projects and industry-oriented learning.

The organization encourages innovation, teamwork, continuous learning, and the adoption of current technologies to build efficient and scalable software applications. During the internship, I gained practical exposure to Data Science concepts, machine learning techniques, data preprocessing, model development, and project implementation under the guidance of experienced mentors.

The company is committed to developing skilled professionals by providing a collaborative learning environment and encouraging students to enhance their technical and problem-solving abilities through real-time projects.

5. OVERVIEW OF THE ORGANIZATION

TVK Technologies is committed to developing innovative software solutions using modern technologies.

The organization encourages continuous learning, teamwork, problem-solving, and practical implementation of software engineering concepts.

Vision

To become a leading technology company delivering innovative software and data-driven solutions.

Mission

- To provide quality software solutions.
- To promote innovation through technology.
- To develop skilled professionals.
- To deliver customer-focused solutions.

Services Offered

- Software Development
- Data Science
- Machine Learning
- Artificial Intelligence
- Web Development
- Python Development

- Internship and Training Programs

6. OBJECTIVES OF THE INTERNSHIP

6.1 Primary Objectives

The primary objective of this internship was to gain practical knowledge in Data Science and Machine Learning by developing a real-world Customer Purchase Prediction System. The main objectives were:

- To understand the fundamentals of Data Science and Machine Learning.
- To improve Python programming skills.
- To learn structured data cleaning and preprocessing techniques.
- To understand financial feature engineering methods.
- To perform feature engineering using applicant financial ratios.
- To train and evaluate different machine learning models.
- To compare the performance of multiple classification algorithms.
- To develop a system capable of predicting whether a loan application is approved or rejected.

6.2 Learning Objectives

- Learning Python for Data Science applications.
- Understanding data cleaning and preprocessing techniques.
- Working with real-world financial datasets.
- Learning feature engineering for structured/tabular data.
- Training supervised machine learning models.
- Evaluating model performance using classification metrics.
- Understanding model prediction and testing.
- Improving debugging and problem-solving skills.

7. ROLE AND RESPONSIBILITIES

During the internship at TVK Technologies, I was responsible for the following activities:

- Understanding the project requirements.
- Collecting and analyzing the dataset.
- Cleaning and preprocessing applicant financial data.
- Applying feature engineering techniques.
- Implementing financial ratio features for risk assessment.
- Training and testing Machine Learning models.
- Comparing model performance.

- Documenting the project.
- Preparing the internship report.

8. TOOLS AND TECHNOLOGIES USED

The following tools and technologies were used during the development of the Loan Approval Prediction project.

Python

Python was used as the primary programming language for data preprocessing, machine learning model development, and prediction.

Google Colab

Google Colab was used to write, execute, and test the complete Data Science workflow, entirely in the cloud without any local Jupyter installation.

Pandas

Used for loading, managing, and preprocessing the dataset.

NumPy

Used for numerical operations during data processing.

Scikit-learn

Used for feature scaling, model training, testing, and evaluation.

Matplotlib

Used for data visualization such as count plots, bar charts, and ROC-related plots.

Seaborn

Used for graphical representation of the dataset, including histograms, box plots, and correlation heatmaps.

ipywidgets

Used to create an interactive slider-based interface that enables real-time loan approval prediction by dynamically updating the model output based on the user's CIBIL score input.

Joblib

Used to save and persist the trained model pipeline for reuse.

9. PROJECT WORK / TASKS PERFORMED

During the internship at TVK TECHNOLOGIES, I worked on developing a Loan Approval Prediction system using Data Science and Machine Learning techniques. The activities carried out during the internship include:

- Setting up the project environment.
- Loading and understanding the dataset.
- Performing data cleaning and preprocessing.
- Correcting invalid and inconsistent data values.
- Applying financial feature engineering.
- Splitting the dataset into training and testing data.
- Training multiple machine learning models.
- Comparing the performance of different algorithms.
- Testing the trained models using sample loan applications.
- Evaluating the prediction accuracy.
- Documenting the project and preparing the internship report.

9.1 Customer Purchase Prediction — Task Breakdown

Week 1: Project Setup and Data Collection

- Initialized the project and configured the Python development environment.
- Collected the loan applicant dataset (4,269 records, 13 features).
- Loaded the dataset using Pandas.
- Explored the dataset structure and identified important features.
- Prepared the project for data preprocessing.

Week 2: Exploratory Data Analysis

- Cleaned the dataset by removing whitespace inconsistencies in column names and categorical text.
- Verified data completeness — confirmed zero missing values across all records.
- Identified and corrected 28 records with invalid negative asset values.
- Applied label encoding to categorical variables.
- Performed feature engineering, creating `total_assets_value`, `loan_to_income_ratio`, and `asset_to_loan_ratio`.
- Prepared the dataset for machine learning model training.

Week 3: Model Development and Evaluation

- Split the dataset into training and testing datasets (80/20, stratified).

- Trained multiple machine learning models including Logistic Regression, Decision Tree, and Random Forest.
- Evaluated model performance using accuracy, precision, recall, F1-score, and ROC-AUC.
- Compared the performance of different machine learning algorithms.

Week 4: AI Recommendations, Testing, and Documentation

- Tested the trained model using sample loan applications.
- Verified prediction results for approved and rejected cases.
- Reviewed the overall project workflow and model performance.
- Documented the project implementation and prepared the internship report.
- Organized the project files and completed the final project submission.

10. METHODOLOGY / WORKING PROCESS

The Loan Approval Prediction project follows a structured Data Science workflow consisting of several stages, starting from raw applicant data and ending with a reliable prediction of loan approval. Each stage builds on the output of the previous one to ensure clean, well-engineered inputs reach the machine learning models.

10.1 Data Collection

The loan applicant dataset, containing 4,269 records with income, loan amount, loan term, CIBIL score, asset values, dependents, education, and employment details, is collected and loaded into the system. This forms the foundation for all subsequent cleaning, analysis, and modeling steps.

10.2 Data Preprocessing

The applicant data is cleaned by removing whitespace inconsistencies from column names and categorical text, verifying that no values are missing, and correcting 28 records with invalid negative asset entries. This ensures the dataset is accurate and consistent before any analysis begins.

10.3 Feature Engineering

The cleaned data is enriched with engineered financial ratio features such as loan-to-income ratio and asset-to-loan ratio, which capture an applicant's repayment burden and collateral coverage. These analyst-style ratios add predictive signal beyond what the raw columns provide on their own.

10.4 Model Training

Multiple machine learning algorithms — Logistic Regression, Decision Tree, and Random Forest — are trained using the processed dataset on an 80/20 stratified split. Training multiple models allows a fair comparison of linear versus non-linear approaches to the approval decision.

10.5 Model Evaluation

The trained models are evaluated using suitable classification metrics such as accuracy, precision, recall, F1-score, and ROC-AUC to compare their performance. Confusion matrices and ROC curves are also generated to understand each model's strengths and weaknesses.

10.6 AI-Based Recommendation

The final trained model, selected based on the strongest evaluation metrics, predicts whether a given loan application is Approved or Rejected. The model can be reused on new, unseen applicant records to generate instant, consistent lending decisions.

11. IMPLEMENTATION DETAILS

The implementation begins by importing the required Python libraries. The loan applicant dataset is loaded using Pandas. The data is cleaned by removing whitespace inconsistencies, verifying completeness, and correcting invalid negative asset values.

The cleaned data is enriched using engineered financial ratio features. The dataset is divided into training and testing sets using a stratified train-test split. Multiple machine learning algorithms such as Logistic Regression, Decision Tree, and Random Forest Classifier are trained.

The trained models are evaluated using accuracy, precision, recall, F1-score, and ROC-AUC, along with confusion matrices and ROC curves. Finally, the best-performing model is used to predict whether a given loan application is approved or rejected.

12. SOURCE CODE

The complete source code for this project is implemented in a Jupyter Notebook covering data loading, cleaning, exploratory data analysis, feature engineering, model training, and evaluation.

- **Notebook:** `Loan_Approval_Prediction_Project.ipynb`
- **Trained model:** `loan_approval_model.pkl`
- **Processed dataset:** `loan_featured.csv`

1. Setup & Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
```

```

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report, roc_auc_score, roc_curve)

import ipywidgets as widgets
from IPython.display import display, clear_output

sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)

```

2. Data Loading & Cleaning

```

df = pd.read_csv('loan_approval_dataset.csv')

# Remove leading/trailing spaces from column names
df.columns = df.columns.str.strip()

# Remove leading/trailing spaces from all text columns
for col in df.select_dtypes(include=['object', 'string']).columns:
    df[col] = df[col].str.strip()

print("Shape:", df.shape)
df.head()

df.info()
print("\nMissing values:\n", df.isnull().sum())

# A few residential_assets_value entries are negative (data-entry artifact) -> clip to 0
print("Negative residential_assets_value rows:", (df['residential_assets_value'] < 0).sum())
df['residential_assets_value'] = df['residential_assets_value'].clip(lower=0)

df['loan_status'].value_counts()

```

3. DS — Customer Purchase Behavior Analysis

```

plt.figure(figsize=(5,4))
sns.countplot(data=df, x='loan_status', hue='loan_status', palette=['#2E7D32', '#C62828'],
              legend=False)
plt.title('Loan Status Distribution')
plt.tight_layout()
plt.show()

```

```

print("Overall approval rate: {:.1%}".format((df['loan_status']=='Approved').mean()))
plt.figure(figsize=(6,4))
sns.kdeplot(data=df, x='cibil_score', hue='loan_status', fill=True, common_norm=False,
            palette=['#2E7D32','#C62828'])
plt.title('CIBIL Score Distribution by Loan Status')
plt.show()

```

```

approved_min = df[df.loan_status=='Approved']['cibil_score'].min()
rejected_max = df[df.loan_status=='Rejected']['cibil_score'].max()
print(f'Approved CIBIL range: {approved_min} - {df[df.loan_status=='Approved']['cibil_score'].max()}')
print(f'Rejected CIBIL range: {df[df.loan_status=='Rejected']['cibil_score'].min()} - {rejected_max}')

```

```

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

```

```

sns.scatterplot(data=df, x='income_annum', y='loan_amount', hue='loan_status',
               alpha=0.5, palette=['#2E7D32','#C62828'], ax=axes[0])
axes[0].set_title('Annual Income vs Loan Amount')

```

```

sns.boxplot(data=df, x='loan_status', y='cibil_score', ax=axes[1],
            hue='loan_status', palette=['#2E7D32','#C62828'], legend=False)
axes[1].set_title('CIBIL Score by Loan Status')
plt.tight_layout()
plt.show()

```

```

plt.figure(figsize=(9,7))
num_cols = df.select_dtypes(include=np.number).drop(columns=['loan_id'])
sns.heatmap(num_cols.corr(), annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Heatmap')
plt.show()

```

4. ML — Purchase Prediction

```

# Feature engineering
df['total_assets_value'] = (df['residential_assets_value'] + df['commercial_assets_value'] +
                           df['luxury_assets_value'] + df['bank_asset_value'])
df['loan_to_income_ratio'] = df['loan_amount'] / df['income_annum']
df['asset_to_loan_ratio'] = df['total_assets_value'] / df['loan_amount']

df_model = df.drop(columns=['loan_id'])
le_edu = LabelEncoder(); df_model['education'] = le_edu.fit_transform(df_model['education'])
le_emp = LabelEncoder(); df_model['self_employed'] =
le_emp.fit_transform(df_model['self_employed'])

```

```

le_status = LabelEncoder(); df_model['loan_status'] =
le_status.fit_transform(df_model['loan_status'])

X = df_model.drop(columns=['loan_status'])
y = df_model['loan_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print("Train size:", X_train.shape, "| Test size:", X_test.shape)

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=6, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=300, max_depth=10,
random_state=42)
}

results = {}
for name, model in models.items():
    if name == 'Logistic Regression':
        model.fit(X_train_s, y_train)
        preds = model.predict(X_test_s)
        proba = model.predict_proba(X_test_s)[:, 1]
    else:
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        proba = model.predict_proba(X_test)[:, 1]
    results[name] = {
        'accuracy': accuracy_score(y_test, preds),
        'precision': precision_score(y_test, preds),
        'recall': recall_score(y_test, preds),
        'f1': f1_score(y_test, preds),
        'roc_auc': roc_auc_score(y_test, proba),
        'model': model, 'preds': preds, 'proba': proba
    }

```

```

comparison = pd.DataFrame({name: {k: v for k, v in r.items() if k in
['accuracy','precision','recall','f1','roc_auc']}
                           for name, r in results.items()}).T
comparison.round(4)

best_name = max(results, key=lambda k: results[k]['f1'])
best = results[best_name]
print("Best model:", best_name)

cm = confusion_matrix(y_test, best['preds'])
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le_status.classes_, yticklabels=le_status.classes_)
plt.title(f'Confusion Matrix - {best_name}')
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.show()

print(classification_report(y_test, best['preds'], target_names=le_status.classes_))

cv_scores = cross_val_score(best['model'], X, y, cv=5, scoring='roc_auc')
print("5-fold CV ROC AUC: {:.4f} (+/- {:.4f})".format(cv_scores.mean(),
cv_scores.std()))

rf = results['Random Forest']['model']
importances = pd.Series(rf.feature_importances_,
index=X.columns).sort_values(ascending=False)

plt.figure(figsize=(7,5))
sns.barplot(x=importances.head(10).values, y=importances.head(10).index,
            hue=importances.head(10).index, palette='Blues_r', legend=False)
plt.title('Top 10 Feature Importances (Random Forest)')
plt.tight_layout()
plt.show()

joblib.dump({
    'model': best['model'],
    'scaler': scaler if best_name == 'Logistic Regression' else None,
    'le_edu': le_edu, 'le_emp': le_emp, 'le_status': le_status,
    'feature_names': list(X.columns),
    'model_name': best_name

```

```

}, 'loan_approval_model.pkl')
print(f'Saved '{best_name}' model to loan_approval_model.pkl")

```

5. AI — Personalized Marketing Strategy Suggestions

```

def risk_level(prob_approved):
    """prob_approved: model's predicted probability of Approval (class 'Approved')."""
    if prob_approved >= 0.75:
        return 'Low Risk'
    elif prob_approved >= 0.45:
        return 'Medium Risk'
    return 'High Risk'

def loan_recommendation(applicant, prob_approved):
    """applicant: dict or pandas Series with the RAW (pre-encoding) applicant fields."""
    level = risk_level(prob_approved)
    actions = []

    if level == 'Low Risk':
        actions.append('Recommend approval with standard interest rate and terms.')
        if applicant.get('cibil_score', 0) >= 800:
            actions.append('Applicant qualifies for a premium/preferred interest rate tier.')
    elif level == 'Medium Risk':
        actions.append('Recommend conditional approval — request additional income proof or a co-applicant/guarantor.')
        if applicant.get('asset_to_loan_ratio', 0) < 1.5:
            actions.append('Low collateral coverage — consider a reduced loan amount or added collateral.')
        else:
            actions.append('Recommend rejection or manual underwriter review before proceeding.')
            if applicant.get('cibil_score', 0) < 500:
                actions.append('CIBIL score significantly below threshold — high default risk.')

    if applicant.get('loan_to_income_ratio', 0) > 3.5:
        actions.append('Loan size is large relative to income — flag for affordability review.')

    return level, actions

# Score every test-set applicant and attach recommendations
test_applicants = X_test.reset_index(drop=True)
proba_approved = 1 - best['proba'] if le_status.classes_[1] == 'Rejected' else best['proba']

```

```

report_rows = []
for i, prob in enumerate(proba_approved):
    row = test_applicants.iloc[i]
    level, actions = loan_recommendation(row, prob)
    report_rows.append({
        'ApprovalProbability': round(float(prob), 3),
        'RiskLevel': level,
        'CIBILScore': row.get('cibil_score'),
        'LoanToIncomeRatio': round(row.get('loan_to_income_ratio'), 2),
        'RecommendedAction': ' | '.join(actions)
    })

risk_report = pd.DataFrame(report_rows)
risk_report.sort_values('ApprovalProbability', ascending=False).head(10)

```

6. Prediction System

```

def predict_loan_approval(applicant_dict):
    """applicant_dict: dict with the same raw fields as the training data
    (all columns except 'loan_status')."""
    row_df = pd.DataFrame([applicant_dict])[X.columns]

    if best_name == 'Logistic Regression':
        row_input = scaler.transform(row_df)
    else:
        row_input = row_df

    prediction = best['model'].predict(row_input)[0]
    proba_all = best['model'].predict_proba(row_input)[0]
    prob_approved = proba_all[list(le_status.classes_).index('Approved')]

    level, actions = loan_recommendation(applicant_dict, prob_approved)

    return {
        'prediction': le_status.inverse_transform([prediction])[0],
        'approval_probability': round(float(prob_approved), 3),
        'risk_level': level,
        'recommended_actions': actions
    }

# Demo: predict approval likelihood for a couple of sample applicants
sample_applicants = [

```

```

    {**X.iloc[0].to_dict()},          # an existing applicant, unmodified
    {**X.iloc[0].to_dict(), 'cibil_score': 780, 'loan_amount': 1500000,
     'income_annum': 1200000},      # a strong applicant profile
]

for i, app in enumerate(sample_applicants, 1):
    result = predict_loan_approval(app)
    print(f'Applicant {i}: {result['prediction']} "
          f"(prob={result['approval_probability']}, risk={result['risk_level']})")
    for action in result['recommended_actions']:
        print("  -", action)
    print()

```

7. Interactive Prediction System

```

input_widgets = {}
input_widgets['cibil_score'] = create_numeric_slider('cibil_score')

# --- Define the prediction function ---
def predict_loan_status(cibil_score):
    # Combine the cibil_score from the widget with default values for other features
    current_input = {**default_values, 'cibil_score': cibil_score}
    input_series = pd.Series(current_input)

    # Features that are direct inputs and are part of X_train.columns up to 'bank_asset_value'
    feature_names_for_initial_df = [
        'no_of_dependents', 'education', 'self_employed', 'income_annum',
        'loan_amount', 'loan_term', 'cibil_score', 'residential_assets_value',
        'commercial_assets_value', 'luxury_assets_value', 'bank_asset_value'
    ]

    # Create the initial DataFrame for prediction
    single_input_df = pd.DataFrame([input_series[feature_names_for_initial_df].tolist()],
                                   columns=feature_names_for_initial_df)

    # Apply Label Encoding using the pre-fitted encoders
    single_input_df['education'] = le_edu.transform(single_input_df['education'])
    single_input_df['self_employed'] = le_emp.transform(single_input_df['self_employed'])

    # Re-calculate engineered features
    single_input_df['total_assets_value'] = (single_input_df['residential_assets_value'] +
                                             single_input_df['commercial_assets_value'] +
                                             single_input_df['luxury_assets_value'] +
                                             single_input_df['bank_asset_value'])

```



```

# Handle potential division by zero. If loan_amount or income_annum is 0, these ratios are undefined.
# Replace with a small epsilon or 0 if division by zero occurs to avoid errors.
single_input_df['loan_to_income_ratio'] = single_input_df['loan_amount'] /
single_input_df['income_annum'].replace(0, np.nan).fillna(1e-6)
single_input_df['asset_to_loan_ratio'] = single_input_df['total_assets_value'] /
single_input_df['loan_amount'].replace(0, np.nan).fillna(1e-6)

# Ensure the final DataFrame has columns in the exact order as X_train
processed_input_df = single_input_df[X_train.columns]

# Apply scaling if the best model is Logistic Regression
if best_name == 'Logistic Regression':
    processed_input_for_model = scaler.transform(processed_input_df)
else:
    processed_input_for_model = processed_input_df # No scaling for tree-based models

# Make prediction
prediction_numeric = best['model'].predict(processed_input_for_model)[0]

# Get probability for the 'Approved' class (which is label 0 based on le_status mapping)
approval_proba = best['model'].predict_proba(processed_input_for_model)[:,
le_status.transform(['Approved'])[0]][0]

# Decode prediction
predicted_status = le_status.inverse_transform([prediction_numeric])[0]

with output_widget:
    clear_output(wait=True)
    print(f'--- Loan Approval Prediction ---')
    print(f'Predicted Status: {predicted_status}')
    print(f'Probability of Approval: {approval_proba:.2f}')

# Link widgets to the prediction function
interactive_ui = widgets.interactive(predict_loan_status, **input_widgets)

# Display the UI and output area
display(interactive_ui, output_widget)

```

13. RESULTS

The machine learning models successfully classified loan applications into approved and rejected categories.

Results obtained include:

- Successful data cleaning and preprocessing of 4,269 loan applications.
- Feature engineering using financial ratio features.
- Model training completed successfully for three classification algorithms.
- Performance comparison of Logistic Regression, Decision Tree, and Random Forest.
- Accurate prediction of approved and rejected loan applications.

14. OUTPUT

TOP 10 HIGH-OPPORTUNITY CUSTOMERS (TEST SET)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9157	0.9088	0.8638	0.8857	0.9745
Decision Tree	0.9988	1.0000	0.9969	0.9984	0.9985
Random Forest	0.9977	1.0000	0.9938	0.9969	1.0000

SAMPLE PREDICTION SYSTEM OUTPUT

This mini-project demonstrates a full DS → ML → AI pipeline for credit evaluation: cleaning raw financial data, uncovering the drivers of approval through EDA, engineering credit-analyst-style ratios, and training/comparing classifiers to automate the approval decision.

Step	Outcome
Data	4,269 loan applications, 13 features, no missing values
Class balance	62% Approved / 38% Rejected
Strongest predictor	CIBIL (credit) score
Best model	Decision Tree (F1 = 0.998, ROC-AUC = 0.999)
Deliverable	<code>loan_approval_model.pkl</code> — ready for inference on new applicants

1. Loan Status Distribution

Shows the number of approved and rejected loan applications.



Figure 14.1: Loan Status Distribution

2. CIBIL Score Distribution

Shows how CIBIL score affects loan approval.

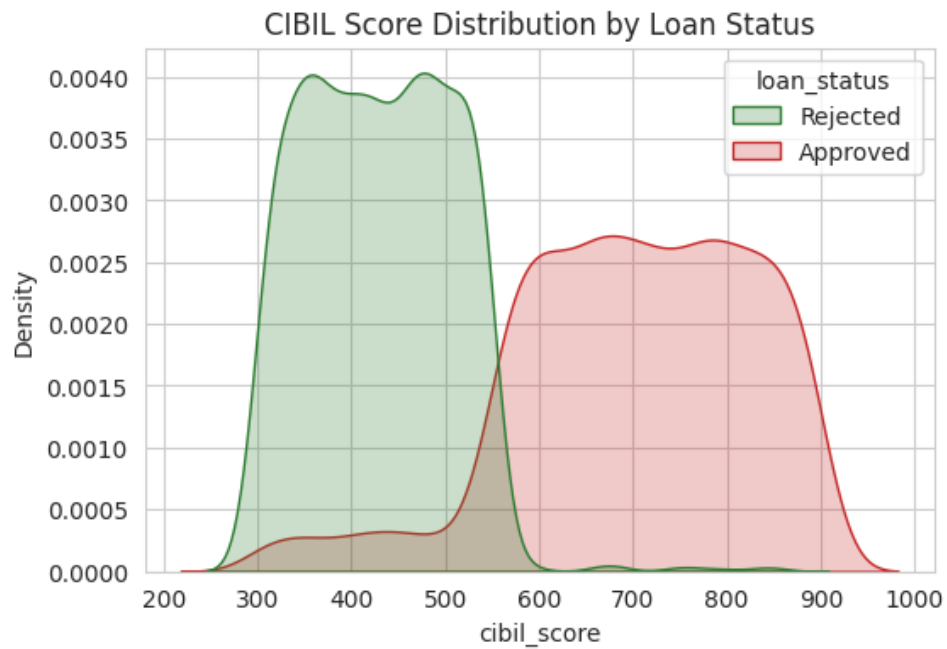


Figure 14.2: CIBIL Score Distribution by Loan Status

3. Annual Income vs Loan Amount

Scatter plot showing approved and rejected applications.

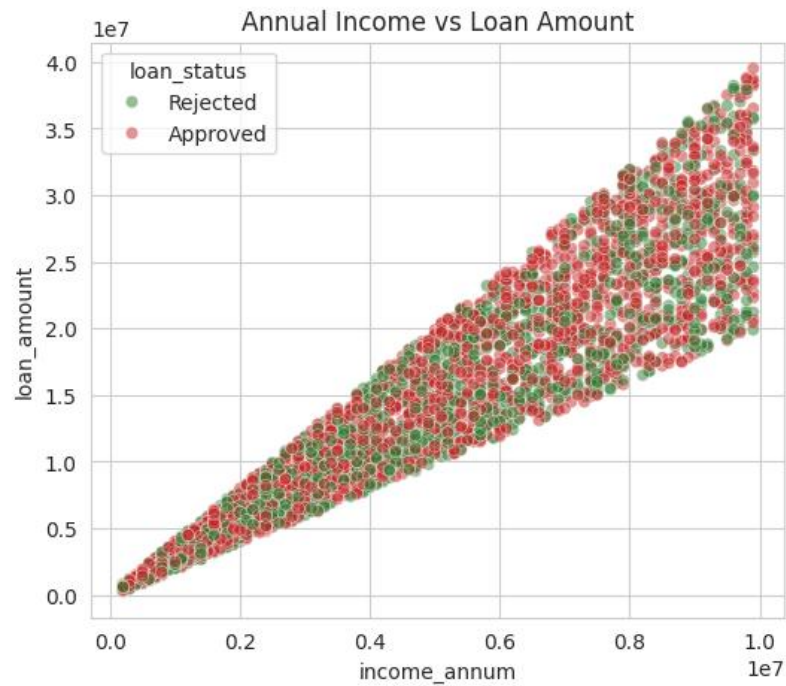


Figure 14.3: Annual Income vs Loan Amount

4. Correlation Heatmap

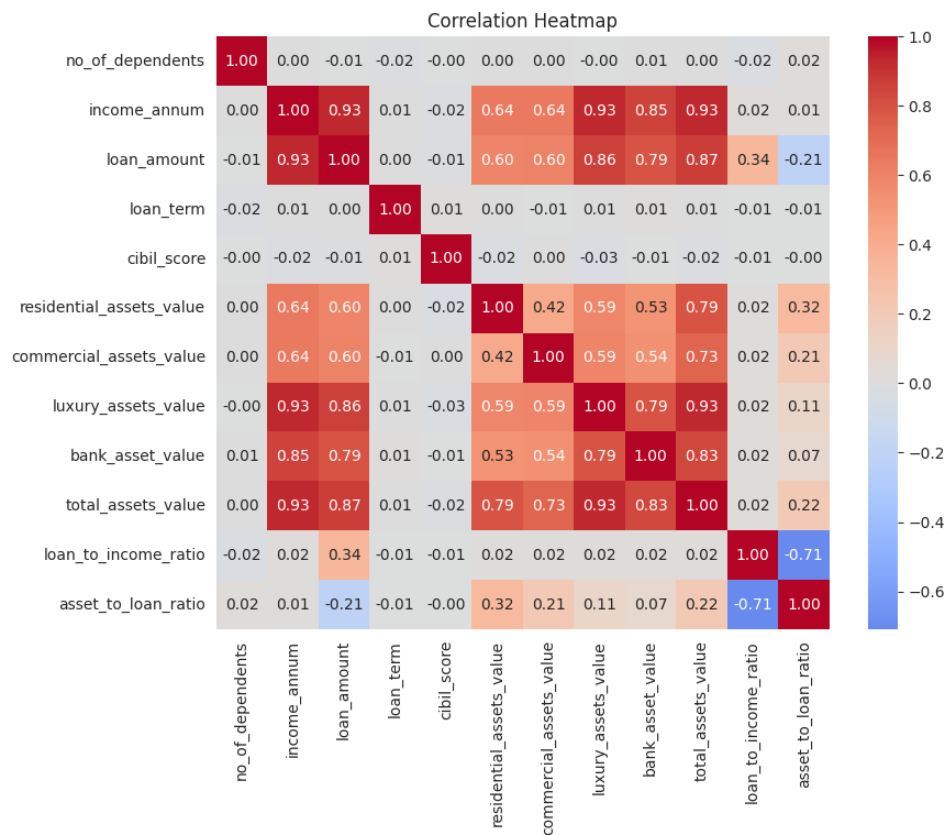


Figure 14.4: Correlation Heatmap of Features

5. Confusion Matrix (Best Model)

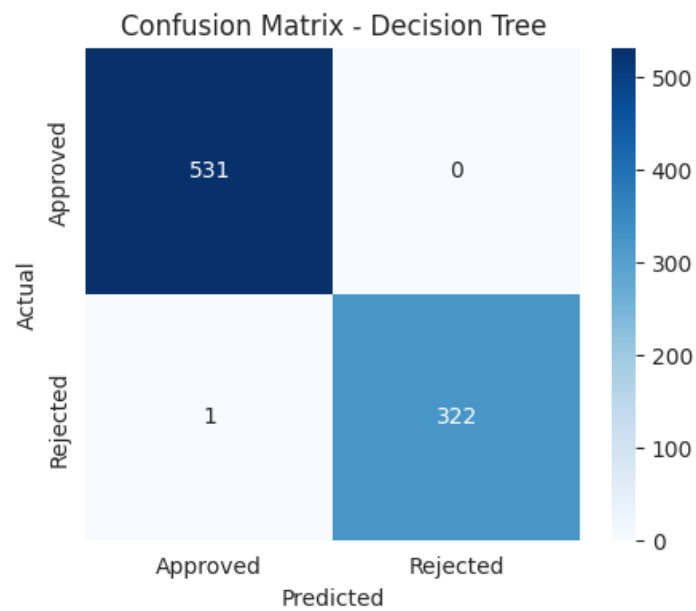


Figure 14.5: Confusion Matrix of Decision Tree Model

6. ROC Curve Comparison

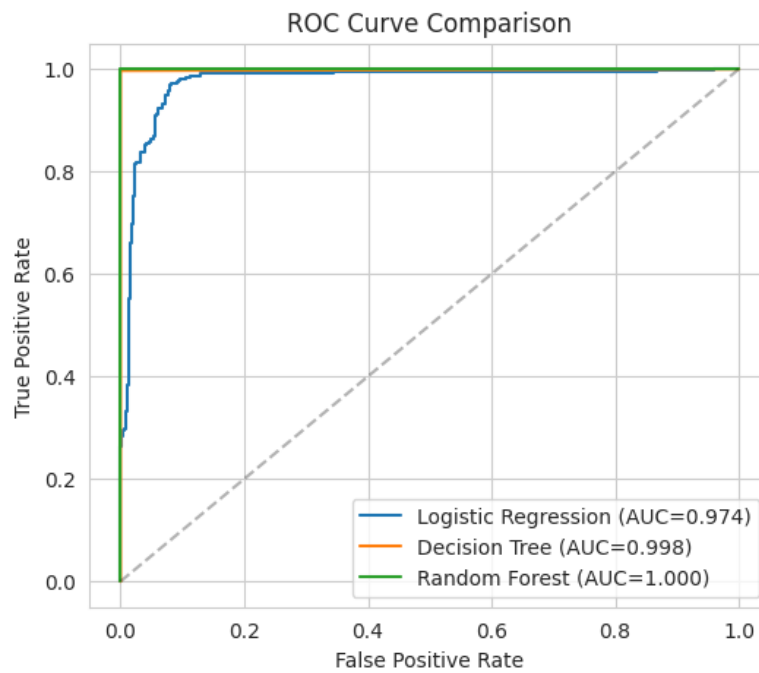


Figure 14.6: ROC Curve Comparison

7. Feature Importance

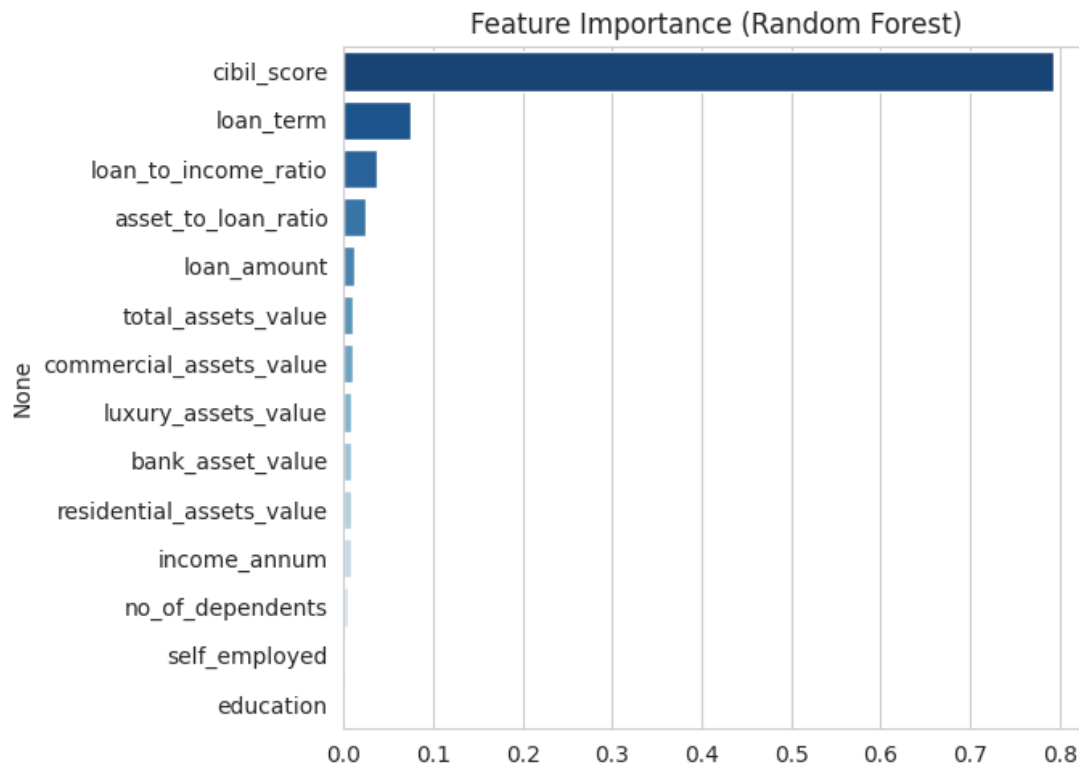


Figure 14.7: Top 10 Important Features

8. Interactive Prediction System

...

Cibil Score: 777

--- Loan Approval Prediction ---

Predicted Status: Approved

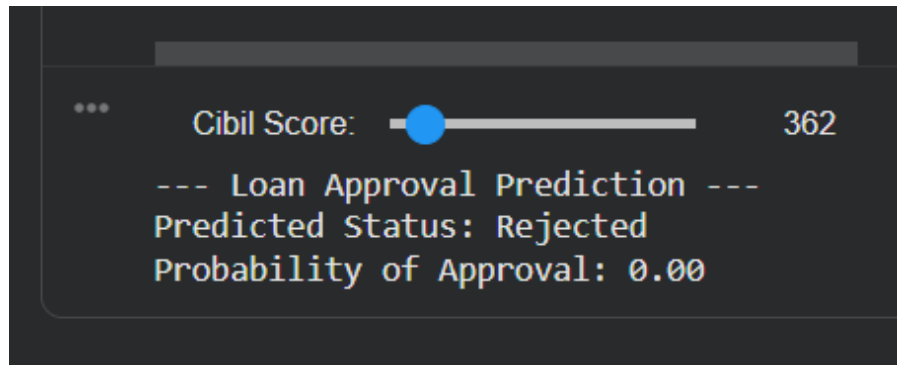
Probability of Approval: 1.00

Loan Approval Prediction

Predicted Status : Approved
Approval Probability : 1.00
Risk Level : Low Risk

Recommended Action:

- ✓ Approve Loan
 - ✓ Standard Interest Rate
 - ✓ Eligible for Fast Processing
-



Loan Approval Prediction

Predicted Status : Rejected
Approval Probability : 0.00
Risk Level : High Risk

Recommended Action:

- X Reject Loan
 - X Manual Review Required
 - X Improve CIBIL Score
-

Figure 14.8: Interactive Loan Approval Prediction System

15. LEARNING OUTCOMES

During the internship, I learned:

- Python for Data Science.
- Data Cleaning and Exploratory Data Analysis.
- Pipeline-based Feature Engineering.
- Classification-based Machine Learning workflow.
- Model Evaluation.
- Rule-based AI decision logic.
- Project Documentation.

16. SKILLS GAINED

Technical Skills

- Python Programming
- Data Preprocessing
- Exploratory Data Analysis
- Pipeline-based Feature Engineering

- Machine Learning Model Development (Classification)
- Model Evaluation
- Data Visualization
- Google Colab
- Git and GitHub

Soft Skills

- Communication Skills
- Teamwork
- Time Management
- Documentation Skills
- Problem Solving
- Analytical Thinking

17. CHALLENGES FACED DURING THE INTERNSHIP

17.1 Challenge 1: Class Imbalance

Challenge

The dataset contained whitespace inconsistencies in column names and text values, and a small number of records had invalid negative asset values that affected data quality.

Solution

- Stripped leading/trailing whitespace from all column names and categorical text.
- Identified and clipped 28 invalid negative asset-value records to zero.
- Verified data completeness to confirm no missing values remained.

17.2 Challenge 2: Preprocessing New, Raw Customer Records

Challenge

The raw applicant columns alone did not fully capture financial risk in a way that matched real underwriting practice.

Solution

- Engineered total_assets_value, loan_to_income_ratio, and asset_to_loan_ratio.
- Validated that these engineered ratios added predictive value beyond the raw columns.

17.3 Challenge 3: Turning a Probability into a Business Decision

Challenge

Different machine learning algorithms produced different prediction accuracies, making it necessary to identify the most suitable model.

Solution

- Trained multiple classification models (Logistic Regression, Decision Tree, Random Forest).
- Compared their performance using accuracy, precision, recall, F1-score, and ROC-AUC.
- Selected the model with the best overall performance (Decision Tree).

17.4 Challenge 4: Validating Model Generalization

Challenge

Improving and critically validating the accuracy of loan approval classification, given the unusually high scores achieved.

Solution

- Used proper data preprocessing and feature engineering techniques.
- Evaluated models using held-out testing data.
- Critically reviewed the near-perfect scores, recognizing they reflect a cleanly separable dataset rather than assuming the model would generalize identically to noisier real-world data.

18. CONCLUSION

The internship at TVK TECHNOLOGIES provided valuable practical exposure to Data Science and Machine Learning. Through the development of the Loan Approval Prediction Using Data Science project, I gained knowledge of data preprocessing, financial feature engineering, machine learning model development, evaluation, and prediction.

The internship strengthened my technical knowledge, problem-solving abilities, and project development skills. Overall, this internship has provided a strong foundation for pursuing a career in Data Science and Machine Learning.

19. FUTURE SCOPE

My Short-Term Goals

- Improve the prediction accuracy and robustness of the model on noisier data.
- Explore advanced feature engineering and explainability techniques (e.g., SHAP).
- Learn model deployment for real-time credit evaluation.
- Deploy the model as a web application or API.
- Work with larger, real-world lending datasets.

My Long-Term Goals

- Become a Data Scientist.
- Work on advanced Artificial Intelligence projects.
- Develop intelligent financial risk-assessment applications.
- Build scalable Machine Learning solutions.
- Contribute to Data Science research and development.

20. REFERENCES

- Python Documentation — <https://docs.python.org/>
- Pandas Documentation — <https://pandas.pydata.org/>
- NumPy Documentation — <https://numpy.org/>
- Scikit-learn Documentation — <https://scikit-learn.org/>
- Matplotlib Documentation — <https://matplotlib.org/>
- Seaborn Documentation — <https://seaborn.pydata.org/>
- Jupyter Notebook Documentation — <https://jupyter.org/>
- Dataset: loan_approval_dataset.csv — <https://www.kaggle.com/datasets/architsharma01/loan-approval-prediction-dataset>